OCR J277 Paper 2: Computational thinking, algorithms and programming

Dougie, these notes cover every point in the OCR specification for Paper 2 (J277/02). Each section starts on a new page and has the same four parts: what you need to know, key terms, exam mistakes that lose marks, and a quick check with answers.

The paper at a glance

- A written paper lasting **1 hour 30 minutes**, worth **80 marks**, which is **50% of your GCSE**.
- You are **not allowed a calculator**.
- It has two sections, **Section A** and **Section B**. You must answer both, and every question is compulsory.
- **Section A (50 marks)** tests your knowledge and understanding of the topics and how you apply them to problems. Exact syntax is not the focus here: you can answer in pseudocode, a flowchart, bullet points, plain English, OCR Exam Reference Language or a high-level language.
- **Section B (30 marks)** tests your practical programming skills: designing, writing, testing and refining programs. OCR's wording: "In Section B, questions assessing students' ability to write or refine algorithms must be answered using either the OCR Exam Reference Language or the high-level programming language they are familiar with." Answers to those questions written in pseudocode, plain English or bullet points get no marks. Small syntax slips are not penalised, but your code must be precise enough to show the logic.
- All code printed in the paper is written in **OCR Exam Reference Language (ERL)**, so you need to read it fluently.

You write code by hand in this exam. There is no computer, no error checker and no auto-indent. Practise writing programs on paper: indent every block clearly, close every block (endif, next i, endwhile) and read your code back line by line. With 80 marks in 90 minutes, a 6-mark question is worth roughly 6 or 7 minutes.

Every code example in these notes is shown twice, side by side: ERL on the left, Python 3 on the right. Learn to switch between them.

ERL cheat sheet

Everything here is taken from the ERL section (3c) of the OCR specification. Arrays and strings are 0 indexed. Arrays store a single data type. Comments start with //.

Comments, variables, constants

```
//This function squares a number
x = 3
name = "Louise"
const vat = 0.2
global userID = "Cust001"
```

Input, output, casting

```
myName = input("Please enter a name")
print("My name is Noni")
str(345)      int("3")
float("4.52") real("4.52")
bool("True")
```

Operators

```
Comparison:  ==  !=  <  <=  >  >=
Arithmetic:  +  -  *  /  MOD  DIV  ^
Boolean:     AND  OR  NOT
```

Selection: if

```
if answer == "Yes" then
    print("Correct")
elseif answer == "No" then
    print("Wrong")
else
    print("Error")
endif
```

Selection: switch (case select)

```
switch day :
    case "Sat":
        print("Saturday")
    case "Sun":
        print("Sunday")
    default:
        print("Weekday")
endswitch
```

Count-controlled loop: for

```
for i=0 to 9          //runs 10 times: 0 to
9
    print("Loop")
next i

for i=2 to 10 step 2  //2, 4, 6, 8, 10
    print(i)
next i

for i=10 to 0 step -1 //10, 9, ..., 0
    print(i)
next i
```

Condition-controlled loops: while, do until

```
while answer != "Correct"  //checks first
    answer = input("New answer")
endwhile

do                         //runs at least
once
    answer = input("New answer")
until answer == "Correct"
```

String handling

```
subject = "ComputerScience"
subject.length          //15
subject.substring(3,5)  //"puter" (start 3,
5 chars)
subject.left(4)         //"Comp"
subject.right(3)        //"nce"
subject.upper           //"COMPUTERSCIENCE"
subject.lower           //"computerscience"
print("Hello, your name is: " + name)
ASC('A')                //65
CHR(97)                 //'a'
```

File handling

```
newFile("myText.txt")   //creates the file
myFile = open("sample.txt")
myFile.writeLine("Add new line") //adds to
END
myFile.readLine()       //returns next line
while NOT myFile.endOfFile()
    print(myFile.readLine())
endwhile
myFile.close()
```

Arrays

```
array colours[5]        //index 0 to 4
array colours = ["Blue", "Pink", "Green",
"Yellow", "Red"]
array gameboard[8,8]    //2D: index 0 to 7
each way
names[3] = "Noni"
gameboard[1,0] = "Pawn"
print(myArray[2,3])
```

Sub programs

```
procedure printName(name)
    print(name)
endprocedure
printName(parameter)    //call

function squared(number)
    squared = number^2
    return squared
endfunction
newValue = squared(4)   //call, store
result
```

Random numbers

```
myVariable = random(1,6)
//random integer 1 to 6 inclusive
myVariable = random(-1.0,10.0)
//random real -1.0 to 10.0 inclusive
```

2.1a Computational thinking and designing algorithms

Specification points 2.1.1 Computational thinking and 2.1.2 Designing, creating and refining algorithms.

What you need to know

Computational thinking is how you think about a problem so that a computer can solve it. OCR names three principles, and you need to understand how each one is used to define a problem (work out what it is) and refine it (improve the solution).

- **Abstraction**: removing detail you do not need, so you can focus on the parts that are needed to solve the problem. A bus route map shows the stops and the order they come in; it leaves out buildings, the exact shape of the roads and the traffic. A dice game program needs a random number from 1 to 6; it does not need the colour or size of a real dice.
- **Decomposition**: breaking a big problem into smaller sub-problems. Each small part is easier to solve, test and share out between programmers, and each part can become its own sub program.
- **Algorithmic thinking**: working out the steps needed to solve the problem and the order they happen in, including where a decision (selection) or a repeat (iteration) is needed. The result is an **algorithm**: a set of steps that solves a problem.

Worked example: a times-table quiz

The problem: "Make a program that asks 10 random times-table questions and tells the user their score."

- **Abstraction**: the program only needs the two numbers in each question, the user's answer and the score. It ignores things such as how long the user takes or their name.
- **Decomposition**: set up the score; make a question; get the answer; check the answer; show the final score.
- **Algorithmic thinking**: set score to 0; repeat 10 times (pick two random numbers from 1 to 12, ask the question, compare the answer, add 1 to score if correct); after the loop, output the score.

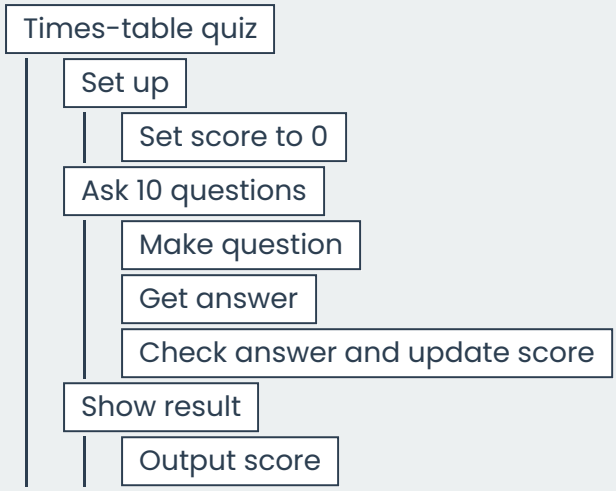
Inputs, processes and outputs. Before designing an algorithm, list what goes in, what the program does with it, and what comes out.

Worked example: inputs, processes and outputs for the quiz

Inputs	Processes	Outputs
The user's answer to each question	Pick two random numbers from 1 to 12; work out the correct answer; compare it with the user's answer; add 1 to the score if correct; repeat 10 times	Each question; "Correct" or "Wrong"; the final score

Structure diagrams show how a problem is decomposed. The whole problem sits at the top. Each level underneath breaks the box above it into smaller parts (subsections), and the lines show which subsection links to which. A structure diagram does not show decisions or the order of loops; that is the job of a flowchart.

Worked example: structure diagram for the quiz (each box links to the box it sits under)



Ways to write an algorithm. You need to create, interpret (explain), correct, complete and refine (improve) algorithms written in each of these:

- **Pseudocode**: a way of writing an algorithm that looks like code but does not have to follow strict rules. It is good for planning. It is accepted in Section A and for "design" questions in Section B, but not for Section B questions that ask you to write or refine code.
- **Flowcharts**: diagrams made of standard symbols joined by arrows.
- **OCR Exam Reference Language (ERL) or a high-level programming language** such as Python. ERL is OCR's own language; every piece of code in the exam paper is written in it. See the cheat sheet.

Flowchart symbols (these are the six in the specification):

Symbol	Shape	What it is for
Terminal	Rectangle with fully rounded ends	Start and Stop (also the start and end of a sub program)
Input/Output	Parallelogram (a slanted rectangle)	Getting data from the user, or outputting data
Process	Rectangle	A calculation or assignment, for example total = total + 1
Decision	Diamond	A question with two exits, labelled Yes and No (or True and False)
Sub program	Rectangle with an extra line down each side	Running a sub program that is defined somewhere else
Line	Arrow	Shows which symbol comes next

Worked example: a password check as a flowchart and as code

Step	Symbol	Text in the symbol	Arrow goes to
1	Terminal	Start	2
2	Input/Output	Input password	3

3	Decision	Is password "Blue42"?	Yes: 5. No: 4
4	Input/Output	Output "Wrong, try again"	Back to 2
5	Input/Output	Output "Welcome"	6
6	Terminal	Stop	

OCR Exam Reference Language

```
password = input("Enter password")
while password != "Blue42"
    print("Wrong, try again")
    password = input("Enter password")
endwhile
print("Welcome")
```

Python 3

```
password = input("Enter password")
while password != "Blue42":
    print("Wrong, try again")
    password = input("Enter password")
print("Welcome")
```

The arrow from step 4 back to step 2 is the loop. The decision in step 3 is the loop's condition.

Nesting means putting one selection or loop inside another: an if inside a for loop, a loop inside a loop, or an if inside an if. Indent each nested block one more level so you can see where it starts and ends. The trace table example below uses an if nested inside a for loop.

Identifying common errors. You need to find errors in code and suggest fixes. A **syntax error** breaks the rules of the language (for example a missing bracket, a misspelt keyword or a missing endif), so the program will not run. A **logic error** lets the program run but gives the wrong result (for example using < instead of <=, dividing by the wrong number or looping the wrong number of times). Section 2.3 covers these in more detail.

Worked example: find and fix the errors

This program should ask for 5 numbers and output their average. It has three errors.

OCR Exam Reference Language

```
total = 0
for i = 1 to 5
    num = input("Enter a number")
    total = total + num
next i
average = total / 4
print("Average: " + average)
```

Python 3

```
total = 0
for i in range(1, 6):
    num = input("Enter a number")
    total = total + num
average = total / 4
print("Average: " + average)
```

Error	Why it is wrong	Fix
num = input("Enter a number")	Input is always a string, so it cannot be added to a number	num = int(input("Enter a number"))
average = total / 4	Logic error: there are 5 numbers, not 4	average = total / 5
print("Average: " + average)	A number cannot be joined to a string without casting	print("Average: " + str(average))

Trace tables let you follow an algorithm by hand to see what it does or to find an error. Make one column for each variable and one for the output. Work through the code one line at a time and write a value in the table every time a variable changes.

Worked example: a fully worked trace table

OCR Exam Reference Language

```
total = 0
for i = 1 to 4
    total = total + i
    if total MOD 2 == 0 then
        print(total)
    endif
next i
```

Python 3

```
total = 0
for i in range(1, 5):
    total = total + i
    if total % 2 == 0:
        print(total)
```

i	total	total MOD 2	Output
	0		
1	1	1	
2	3	1	
3	6	0	6
4	10	0	10

How it works: total starts at 0. Each time round the loop, i is added to total (0 + 1 = 1, 1 + 2 = 3, 3 + 3 = 6, 6 + 4 = 10). The nested if only prints total when it is even (MOD 2 gives 0). The loop runs 4 times because for i = 1 to 4 includes both 1 and 4. The output is 6 then 10.

Key terms

Computational thinking
Thinking about a problem in a way that lets a computer solve it, using abstraction, decomposition and algorithmic thinking.

Abstraction
Removing unnecessary detail from a problem so you can focus on the parts needed to solve it.

Decomposition
Breaking a problem down into smaller sub-problems that are easier to solve.

Algorithmic thinking
Working out the steps needed to solve a problem and the order they must happen in.

Algorithm
A set of steps that solves a problem.

Input
Data that goes into a program, such as something the user types.

Process
Something the program does with data, such as a calculation or a comparison.

Output

Data that comes out of a program, such as text shown on the screen.

Structure diagram
A diagram that shows a problem broken down into subsections and how they link to each other.

Pseudocode
A way of writing an algorithm that looks like code but does not follow the strict rules of a real language.

Flowchart
A diagram of an algorithm made of standard symbols joined by arrows.

OCR Exam Reference Language (ERL)
OCR's own programming language, used for all code in the exam paper.

High-level programming language
A programming language that uses English-like words, such as Python.

Nesting
Putting a selection or loop inside another selection or loop.

Syntax error
An error that breaks the grammar rules of the language, so the program will not run.

Logic error
An error that lets the program run but makes it give an unexpected or wrong result.

Trace table
A table used to follow an algorithm step by step, recording each variable's value and the output.

Refine
To improve an algorithm, for example by making it more efficient or fixing an error.

Exam mistakes that lose marks

- Writing a description or pseudocode when a Section B question says write the code. Those answers get no marks; write ERL or Python.
- Mixing up abstraction and decomposition. Abstraction removes detail; decomposition breaks the problem into parts.
- Drawing a flowchart when asked for a structure diagram. A structure diagram only shows the parts of the problem, not decisions or order.
- Using the wrong flowchart shape: inputs and outputs go in a parallelogram, not a rectangle. Decisions need a diamond with both exits labelled Yes and No.
- Forgetting the arrow that loops back in a flowchart, so the loop never repeats.
- Leaving out endif, next i or endwhile when writing code by hand.
- Off-by-one loop ranges: for i = 1 to 4 runs 4 times (1, 2, 3, 4), and for i = 0 to 4 runs 5 times.
- In trace tables: skipping a row, putting a value in the wrong column, or forgetting to fill in the output column.
- When asked to identify an error, only saying "it is wrong". Give the line, say what is wrong and write the corrected line.

Quick check

1. What is abstraction?
2. Which flowchart symbol is used for a decision, and what must its exits be labelled?
3. A program asks for a temperature in Celsius and outputs it in Fahrenheit. List the input, process and output.
4. Trace this code and give the final value of x: x = 1, then while x < 20, x = x * 3.
5. The pass mark is 50. The code says if score > 50 then print("Pass"). What type of error is this, and how do you fix it?

▼ **Answers**

1. Removing unnecessary detail from a problem so you can focus on the parts needed to solve it.
2. A diamond. Its two exits must be labelled Yes and No (or True and False).
3. Input: the temperature in Celsius. Process: Fahrenheit = Celsius * 9 / 5 + 32. Output: the temperature in Fahrenheit.
4. x goes 1, 3, 9, 27. The loop stops when x is 27, because 27 is not less than 20. Final value: 27.
5. A logic error: a score of exactly 50 would not pass. Fix: if score >= 50 then.

2.1b Searching and sorting algorithms

Specification point 2.1.3 Searching and sorting algorithms.

What you need to know

For each of the five algorithms you need to: understand the main steps and the segments (parts) of code in it; know any pre-requisites (what must be true before you can use it); apply it to a set of data by hand; and recognise it if you are given its code. **You do not need to remember the code**, but reading the code below will help you recognise each algorithm.

Linear search

1. Start at the first item in the list.
2. Compare the item with the value you are looking for (the target).
3. If they match, stop: the item is found.
4. If not, move to the next item and repeat.
5. If you reach the end of the list without a match, the item is not in the list.

Linear search works on any list, sorted or unsorted. It is simple, but on a large list it may have to check every item.

Worked example: linear search for 2 in [7, 3, 9, 2, 5, 8]

Check	Index	Item	Is it 2?
1	0	7	No, move on
2	1	3	No, move on
3	2	9	No, move on
4	3	2	Yes, found at index 3

It took 4 comparisons. If the target was 4, all 6 items would be checked and the result would be "Not found".

OCR Exam Reference Language

```
array nums = [7, 3, 9, 2, 5, 8]
target = int(input("Number to find"))
pos = -1
i = 0
while i <= 5 AND pos == -1
    if nums[i] == target then
        pos = i
    endif
    i = i + 1
endwhile
if pos == -1 then
    print("Not found")
else
    print("Found at index " + str(pos))
endif
```

Python 3

```
nums = [7, 3, 9, 2, 5, 8]
target = int(input("Number to find"))
pos = -1
i = 0
while i <= 5 and pos == -1:
    if nums[i] == target:
        pos = i
    i = i + 1
if pos == -1:
    print("Not found")
else:
    print("Found at index " + str(pos))
```

pos = -1 means "not found yet". The loop stops when the item is found or when every index (0 to 5) has been checked.

Binary search

Pre-requisite: the list must already be sorted. Binary search will give wrong answers on an unsorted list.

1. Set low to the first index and high to the last index.
2. Find the midpoint: `mid = (low + high) DIV 2` (DIV gives the whole-number part, so the midpoint is always a whole index).
3. If the item at mid is the target, stop: found.
4. If the target is bigger than the item at mid, throw away the left half: low = mid + 1.
5. If the target is smaller, throw away the right half: high = mid - 1.
6. Repeat from step 2 while low is less than or equal to high. If low becomes bigger than high, the item is not in the list.

Each check halves the part of the list left to search, so binary search is much faster than linear search on large lists. For a list of 1000 items, binary search needs at most 10 checks; linear search could need 1000.

Worked example: binary search on a sorted list

Index	0	1	2	3	4	5	6	7	8
Item	2	3	5	7	8	9	12	15	18

Search for 15 (found):

Step	low	high	mid = (low + high) DIV 2	Item at mid	Decision
1	0	8	(0 + 8) DIV 2 = 4	8	15 > 8, so low = 5
2	5	8	(5 + 8) DIV 2 = 6	12	15 > 12, so low = 7
3	7	8	(7 + 8) DIV 2 = 7	15	Match: found at index 7

Search for 4 (not found):

Step	low	high	mid = (low + high) DIV 2	Item at mid	Decision
1	0	8	(0 + 8) DIV 2 = 4	8	4 < 8, so high = 3
2	0	3	(0 + 3) DIV 2 = 1	3	4 > 3, so low = 2
3	2	3	(2 + 3) DIV 2 = 2	5	4 < 5, so high = 1
4	2	1			low > high, so stop: not found

OCR Exam Reference Language	Python 3

```
array nums = [2, 3, 5, 7, 8, 9, 12, 15,
18]
target = int(input("Number to find"))
low = 0
high = 8
pos = -1
while low <= high AND pos == -1
    mid = (low + high) DIV 2
    if nums[mid] == target then
        pos = mid
    elseif nums[mid] < target then
        low = mid + 1
    else
        high = mid - 1
    endif
endwhile
if pos == -1 then
    print("Not found")
else
    print("Found at index " + str(pos))
endif
```

```
nums = [2, 3, 5, 7, 8, 9, 12, 15, 18]
target = int(input("Number to find"))
low = 0
high = 8
pos = -1
while low <= high and pos == -1:
    mid = (low + high) // 2
    if nums[mid] == target:
        pos = mid
    elif nums[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
if pos == -1:
    print("Not found")
else:
    print("Found at index " + str(pos))
```

Bubble sort

1. Start at the beginning of the list.
2. Compare the first two items. If they are in the wrong order, swap them.
3. Move along one place and compare the next pair. Keep going to the end of the list. This is one **pass**.
4. After each pass, the largest unsorted item has "bubbled" to its correct place at the end.
5. Repeat passes until a whole pass happens with no swaps. The list is then sorted.

Worked example: bubble sort on [5, 2, 9, 1, 6, 3] (into ascending order)

Pass 1, comparison by comparison:

Compare	Swap?	List after
5 and 2	Yes	[2, 5, 9, 1, 6, 3]
5 and 9	No	[2, 5, 9, 1, 6, 3]
9 and 1	Yes	[2, 5, 1, 9, 6, 3]
9 and 6	Yes	[2, 5, 1, 6, 9, 3]
9 and 3	Yes	[2, 5, 1, 6, 3, 9]

Pass by pass:

Stage	List	Swaps in this pass
Start	[5, 2, 9, 1, 6, 3]	
After pass 1	[2, 5, 1, 6, 3, 9]	4
After pass 2	[2, 1, 5, 3, 6, 9]	2 (5 and 1, then 6 and 3)

After pass 3	[1, 2, 3, 5, 6, 9]	2 (2 and 1, then 5 and 3)
After pass 4	[1, 2, 3, 5, 6, 9]	0, so the list is sorted and the algorithm stops

OCR Exam Reference Language

```
array nums = [5, 2, 9, 1, 6, 3]
do
    swaps = 0
    for i = 0 to 4
        if nums[i] > nums[i + 1] then
            temp = nums[i]
            nums[i] = nums[i + 1]
            nums[i + 1] = temp
            swaps = swaps + 1
        endif
    next i
until swaps == 0
```

Python 3

```
nums = [5, 2, 9, 1, 6, 3]
while True:
    swaps = 0
    for i in range(0, 5):
        if nums[i] > nums[i + 1]:
            temp = nums[i]
            nums[i] = nums[i + 1]
            nums[i + 1] = temp
            swaps = swaps + 1
    if swaps == 0:
        break
```

The loop goes to index 4, not 5, because it compares nums[i] with nums[i + 1]; the last pair is index 4 and index 5. The variable temp holds one value so the swap does not lose it.

Insertion sort

1. Treat the first item as a sorted list of one item.
2. Take the next item from the unsorted part.
3. Compare it with the items in the sorted part, moving left, and shift each bigger item one place to the right.
4. Insert the item into the gap, so the sorted part stays in order.
5. Repeat until every item has been inserted.

Worked example: insertion sort on the same list, [5, 2, 9, 1, 6, 3]

The | separates the sorted part (left) from the unsorted part (right).

Stage	Item inserted	List
Start		[5 | 2, 9, 1, 6, 3]
Pass 1	2 goes before 5	[2, 5 | 9, 1, 6, 3]
Pass 2	9 stays after 5	[2, 5, 9 | 1, 6, 3]
Pass 3	1 goes to the front	[1, 2, 5, 9 | 6, 3]
Pass 4	6 goes between 5 and 9	[1, 2, 5, 6, 9 | 3]
Pass 5	3 goes between 2 and 5	[1, 2, 3, 5, 6, 9]

OCR Exam Reference Language | **Python 3**

```
array nums = [5, 2, 9, 1, 6, 3]
for i = 1 to 5
    current = nums[i]
    pos = i
    while pos > 0 AND nums[pos - 1] >
current
        nums[pos] = nums[pos - 1]
        pos = pos - 1
    endwhile
    nums[pos] = current
next i
```

```
nums = [5, 2, 9, 1, 6, 3]
for i in range(1, 6):
    current = nums[i]
    pos = i
    while pos > 0 and nums[pos - 1] >
current:
        nums[pos] = nums[pos - 1]
        pos = pos - 1
    nums[pos] = current
```

The for loop starts at index 1 because the item at index 0 is already "sorted" on its own. The while loop shifts bigger items right until it finds the gap.

Merge sort

1. Split the list in half, then split each half in half, and keep going until every list has only one item. A list of one item is already sorted.
2. Merge pairs of lists back together. To merge two sorted lists, compare the first item of each, take the smaller one, and repeat until both lists are empty.
3. Keep merging pairs until there is one sorted list.

Worked example: merge sort on [6, 3, 8, 2, 5, 1, 7, 4]

Stage	Lists
Start	[6, 3, 8, 2, 5, 1, 7, 4]
Split 1	[6, 3, 8, 2] [5, 1, 7, 4]
Split 2	[6, 3] [8, 2] [5, 1] [7, 4]
Split 3	[6] [3] [8] [2] [5] [1] [7] [4]
Merge 1	[3, 6] [2, 8] [1, 5] [4, 7]
Merge 2	[2, 3, 6, 8] [1, 4, 5, 7]
Merge 3	[1, 2, 3, 4, 5, 6, 7, 8]

The last merge in detail, joining [2, 3, 6, 8] and [1, 4, 5, 7]:

Compare first items	Take	Merged list so far
2 and 1	1	[1]
2 and 4	2	[1, 2]
3 and 4	3	[1, 2, 3]
6 and 4	4	[1, 2, 3, 4]
6 and 5	5	[1, 2, 3, 4, 5]
6 and 7	6	[1, 2, 3, 4, 5, 6]
8 and 7	7	[1, 2, 3, 4, 5, 6, 7]

Only 8 left	8	[1, 2, 3, 4, 5, 6, 7, 8]

Comparing the algorithms

Algorithm	Pre-requisite	Good points	Bad points
Linear search	None: works on unsorted lists	Simple to write; works on any list	Slow on large lists
Binary search	List must be sorted	Much faster on large lists	The list has to be sorted first; harder to write
Bubble sort	None	Simple to write; sorts within the list without extra memory	Very slow on large lists
Insertion sort	None	Simple; quick on small or nearly sorted lists; no extra memory	Slow on large lists
Merge sort	None	Much faster than the other two on large lists	Uses more memory for the extra lists; harder to write

Recognising an algorithm from its code

- **Linear search**: a loop that checks each index in turn against the target.
- **Binary search**: variables low, high and mid, with (low + high) DIV 2, and low or high moving past mid.
- **Bubble sort**: compares item[i] with item[i + 1], swaps using a temporary variable, and repeats until no swaps are made.
- **Insertion sort**: stores the current item, then a loop that shifts bigger items one place right before putting the current item in the gap.
- **Merge sort**: splits the list into a left half and a right half, then merges them back together by comparing items.

Key terms

Searching algorithm
A set of steps that finds whether, and where, an item is in a list.

Linear search
A search that checks each item in turn, from the start, until it finds the target or reaches the end.

Binary search
A search on a sorted list that checks the middle item and throws away the half that cannot contain the target.

Midpoint
The middle index in binary search, worked out as (low + high) DIV 2.

Pre-requisite
Something that must be true before an algorithm can be used, such as a list being sorted for binary search.

Sorting algorithm

A set of steps that puts items into order, such as ascending (smallest first).

Bubble sort
A sort that compares neighbouring pairs and swaps them if they are in the wrong order, repeating until a pass has no swaps.

Pass
One complete run through the list in a sorting algorithm.

Swap
Exchanging the positions of two items, usually using a temporary variable.

Insertion sort
A sort that takes each item in turn and inserts it into the correct place in a growing sorted part of the list.

Merge sort
A sort that splits a list into single items, then merges them back together in order.

Merge
Joining two sorted lists into one sorted list by repeatedly taking the smaller of their first items.

Exam mistakes that lose marks

- Using binary search on an unsorted list, or forgetting to say "the list must be sorted" when asked for a pre-requisite.
- Saying linear search needs a sorted list. It does not.
- Working out the midpoint wrongly. Use mid = (low + high) DIV 2, so (0 + 3) DIV 2 is 1, not 1.5 or 2.
- Setting low = mid or high = mid instead of mid + 1 or mid - 1. The midpoint has already been checked.
- Showing only the final sorted list when the question asks for each pass. Show the whole list after every pass.
- Stopping bubble sort too early: it needs one complete pass with no swaps to know the list is sorted.
- Mixing up bubble sort and insertion sort. Bubble sort swaps neighbouring pairs across the whole list; insertion sort builds a sorted part from the left.
- In merge sort, skipping the split stages, or merging by just joining lists without comparing their first items.
- Saying merge sort uses less memory. It uses more, because it creates extra lists.

Quick check

1. What must be true about a list before you can use binary search on it?
2. Using binary search on [1, 4, 6, 9, 13, 17, 20], which items are checked, in order, when searching for 17?
3. Write down [4, 1, 3, 2] after the first pass of bubble sort.
4. How does bubble sort know when it has finished?
5. An algorithm takes each item in turn and puts it into the correct place in a sorted section at the start of the list. Name it.

▼ **Answers**

1. It must be sorted.
2. low = 0, high = 6, mid = (0 + 6) DIV 2 = 3, item 9; 17 > 9 so low = 4. Then mid = (4 + 6) DIV 2 = 5, item 17: found. Items checked: 9, then 17.
3. 4 and 1 swap: [1, 4, 3, 2]. 4 and 3 swap: [1, 3, 4, 2]. 4 and 2 swap: [1, 3, 2, 4]. Answer: [1, 3, 2, 4].
4. When it completes a whole pass with no swaps.
5. Insertion sort.

2.2a Programming fundamentals

Specification point 2.2.1 Programming fundamentals.

What you need to know

Variables are named places in memory that hold a value which can change while the program runs. **Constants** hold a value that cannot change while the program runs. Using a constant means the value cannot be changed by mistake, and if it ever needs updating (for example a new VAT rate) you change it in one place.

Assignment gives a variable a value, using a single equals sign: `score = 0`. Assignment works right to left: `score = score + 1` takes the old score, adds 1, and stores the result back in score.

In ERL a constant is written with `const`. Python has no const keyword; programmers write constants in capital letters so everyone knows not to change them.

Operators

ERL	Python	Meaning	Example	Result
+	+	Addition	17 + 5	22
-	-	Subtraction	17 - 5	12
*	*	Multiplication	17 * 5	85
/	/	Division	17 / 5	3.4
MOD	%	Modulo: the remainder after dividing	17 MOD 5	2
DIV	//	Quotient: the whole number of times it divides	17 DIV 5	3
^	**	Exponentiation (to the power)	2 ^ 4	16

Comparison operator (same in ERL and Python)	Meaning
==	Equal to
!=	Not equal to
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to

MOD is useful for checking whether a number is even (`num MOD 2 == 0`) or whether one number divides exactly into another. DIV and MOD together split a total into whole groups and a remainder.

Inputs and outputs. `input()` shows a message and waits for the user to type something. Whatever they type comes back as a **string**, so cast it with `int()` or `float()` if you need a number. `print()` outputs to the screen.

> **Worked example: constants, operators, inputs and outputs (minutes into hours)**

OCR Exam Reference Language	Python 3

`const minutesInHour = 60`
`totalMinutes = int(input("Enter the number of minutes"))`
`hours = totalMinutes DIV minutesInHour`
`minutes = totalMinutes MOD minutesInHour`
`print(str(hours) + " hours and " + str(minutes) + " minutes")`
	`MINUTES_IN_HOUR = 60`
	`totalMinutes = int(input("Enter the number of minutes"))`
	`hours = totalMinutes // MINUTES_IN_HOUR`
	`minutes = totalMinutes % MINUTES_IN_HOUR`
	`print(str(hours) + " hours and " + str(minutes) + " minutes")`

If the user enters 135: 135 DIV 60 = 2 and 135 MOD 60 = 15, so the output is "2 hours and 15 minutes".

The three programming constructs control the flow of every program:

- **Sequence**: instructions run one after another, in order. The minutes example above is pure sequence.
- **Selection**: the program chooses which instructions to run, using if, elseif, else, or switch (case select).
- **Iteration**: the program repeats instructions using a loop. A **count-controlled** loop (for) repeats a set number of times. A **condition-controlled** loop (while, or do until) repeats until a condition changes, so you use it when you do not know in advance how many times it will run.

Worked example: selection with if, elseif and else

OCR Exam Reference Language	Python 3

`mark = int(input("Enter mark"))`
`if mark >= 70 then`
`  print("Distinction")`
`elseif mark >= 50 then`
`  print("Pass")`
`else`
`  print("Fail")`
`endif`
	`mark = int(input("Enter mark"))`
	`if mark >= 70:`
	`  print("Distinction")`
	`elif mark >= 50:`
	`  print("Pass")`
	`else:`
	`  print("Fail")`

The conditions are checked from the top. A mark of 55 fails the first test (not 70 or more), passes the second, outputs "Pass" and skips the rest. Order is important: if the test for 50 came first, a mark of 85 would output "Pass".

Worked example: selection with switch (case select)

OCR Exam Reference Language	Python 3 (3.10 or later)

`day = input("Enter a day")`
`switch day :`
`    case "Sat":`
`        print("Saturday")`
`    case "Sun":`
`        print("Sunday")`
`    default:`
`        print("Weekday")`
`endswitch`
	`day = input("Enter a day")`
	`match day:`
	`    case "Sat":`
	`        print("Saturday")`
	`    case "Sun":`
	`        print("Sunday")`
	`    case _:`
	`        print("Weekday")`

Switch compares one variable with several possible values. default (case _ in Python) runs when none of the cases match. In older versions of Python, write the same thing with if, elif and else.

Worked example: count-controlled loops (for)

OCR Exam Reference Language	Python 3

`for i = 0 to 9`
`    print("Loop")`
`next i`
 	
`for i = 2 to 10 step 2`
`    print(i)`
`next i`
 	
`for i = 10 to 0 step -1`
`    print(i)`
`next i`
	`for i in range(0, 10):`
	`    print("Loop")`
	
	`for i in range(2, 11, 2):`
	`    print(i)`
	
	`for i in range(10, -1, -1):`
	`    print(i)`

The first loop prints "Loop" 10 times (i = 0 to 9). The second prints 2, 4, 6, 8, 10. The third counts down from 10 to 0. In ERL the end number is included. In Python's range, the end number is **not** included, which is why range goes to 10, 11 and -1.

Worked example: condition-controlled loops (while and do until)

OCR Exam Reference Language	Python 3

`answer = ""`
`while answer != "Correct"`
`    answer = input("New answer")`
`endwhile`
 	
`do`
`    answer = input("New answer")`
`until answer == "Correct"`
	`answer = ""`
	`while answer != "Correct":`
	`    answer = input("New answer")`
	
	`while True:`
	`    answer = input("New answer")`
	`    if answer == "Correct":`
	`        break`

A **while** loop checks its condition **before** each run, so it might not run at all. A **do until** loop runs once and then checks, so it always runs **at least once**. Notice the conditions are opposites: while keeps going while the answer is not "Correct"; do until keeps going until it is. Python has no do until loop, so you use while True and break out when the condition is met.

Boolean operators join or reverse conditions:

- **AND**: true only if both conditions are true.
- **OR**: true if at least one condition is true.
- **NOT**: reverses a condition, so true becomes false and false becomes true.

Worked example: AND, OR and NOT in a program

OCR Exam Reference Language

```
day = input("Enter the day")
age = int(input("Enter your age"))
if day == "Sat" OR day == "Sun" then
    print("Weekend opening times")
endif
if age >= 12 AND age <= 16 then
    print("Teen ticket")
endif
if NOT (age >= 18) then
    print("You must come with an adult")
endif
```

Python 3

```
day = input("Enter the day")
age = int(input("Enter your age"))
if day == "Sat" or day == "Sun":
    print("Weekend opening times")
if age >= 12 and age <= 16:
    print("Teen ticket")
if not (age >= 18):
    print("You must come with an adult")
```

With day "Sat" and age 14, all three messages are output. Each side of AND and OR must be a full comparison: write `day == "Sat" OR day == "Sun"`, never `day == "Sat" OR "Sun"`.

Key terms

Variable
A named place in memory that holds a value which can change while the program runs.

Constant
A named value that cannot change while the program runs.

Assignment
Giving a variable a value, using =.

Operator
A symbol or word that carries out an operation on values, such as + or MOD.

Arithmetic operator
An operator that does a calculation: +, -, *, /, MOD, DIV or ^.

Comparison operator
An operator that compares two values and gives true or false: ==, !=, <, <=, > or >=.

MOD (modulo)
Gives the remainder after division, so 17 MOD 5 is 2.

DIV (quotient)
Gives the whole-number result of a division, so 17 DIV 5 is 3.

Exponentiation (^)
Raises a number to a power, so 2 ^ 4 is 16.

Input
Data the program takes in, such as text typed by the user with input().

Output
Data the program gives out, such as text shown with print().

Sequence

Instructions running one after another in order.

Selection
Choosing which instructions to run based on a condition, using if or switch.

Iteration
Repeating instructions using a loop.

Count-controlled loop
A loop (for) that repeats a set number of times.

Condition-controlled loop
A loop (while or do until) that repeats until a condition is met.

Boolean operator
AND, OR or NOT, used to combine or reverse conditions.

Exam mistakes that lose marks

- Confusing = with ==. Use = to assign a value and == to compare: `if score == 10 then`, not `if score = 10 then`.
- Forgetting to cast input to an integer. `age = input("Age")` gives a string; write `age = int(input("Age"))`.
- Off-by-one loop ranges. ERL `for i = 0 to 9` runs 10 times and includes 9. Python `range(0, 9)` stops at 8. Count the loops in your head before you write the numbers.
- Using ^ for powers in Python. In Python ^ does something else; use **.
- Mixing up MOD and DIV. MOD is the remainder; DIV is the whole-number part.
- Forgetting to set a total or counter to 0 before the loop starts.
- A while loop whose condition never changes inside the loop, so it loops forever.
- Using AND where you need OR: `age < 11 AND age > 16` can never be true. To reject ages outside 11 to 16, use OR.
- Writing the elseif conditions in the wrong order, so a later condition can never be reached.
- Choosing a for loop when you do not know how many times it will repeat. Use while or do until instead.

Quick check

1. What are 23 MOD 4 and 23 DIV 4?
2. What is 3 ^ 2?
3. How many times does the loop for i = 3 to 7 run?
4. What is the difference between a while loop and a do until loop?
5. Write the Python version of the ERL loop for i = 1 to 5.

▼ **Answers**

1. 23 MOD 4 = 3 (the remainder). 23 DIV 4 = 5 (4 goes into 23 five whole times).
2. 9.
3. 5 times (3, 4, 5, 6 and 7).
4. A while loop checks its condition before it runs, so it might run 0 times. A do until loop checks after it runs, so it always runs at least once.

5. for i in range(1, 6):

2.2b Data types and additional programming techniques

Specification points 2.2.2 Data types and 2.2.3 Additional programming techniques.

What you need to know

Data types tell the computer what kind of data a variable holds. You need to choose a suitable data type for data in a given scenario.

Data type	Python name	What it holds	Examples	Good for
Integer	int	Whole numbers	42, -7, 0	Age, number of lives, score
Real	float	Numbers with a decimal part	3.14, -0.5, 4.99	Price, height in metres
Boolean	bool	Only two values: True or False	True, False	Has paid, is logged in
Character	str (one character)	A single letter, digit or symbol	'A', '7', '?'	A grade, an initial
String	str	A sequence of characters	"Dougie", "07700 900123"	Names, postcodes, phone numbers

A phone number is stored as a **string**, not an integer: it can start with 0 (which an integer would drop), it can contain spaces, and you never do maths with it.

Casting temporarily changes a value from one data type to another. It is useful when you need to do maths with something typed in (input always gives a string), or when you need to join a number onto a string for output.

Worked example: casting

OCR Exam Reference Language

```
num = "45"
total = int(num) + 5
print("Total: " + str(total))
price = float("4.52")
weight = real("4.52")
paid = bool("True")
```

Python 3

```
num = "45"
total = int(num) + 5
print("Total: " + str(total))
price = float("4.52")
weight = float("4.52")
paid = bool("True")
```

int("45") turns the string "45" into the integer 45, so 45 + 5 = 50. str(total) turns 50 back into a string so it can be joined to "Total: ". ERL accepts both float() and real() for real numbers; Python only has float().

String manipulation. Characters in a string are numbered from 0. You need to know **concatenation** (joining strings with +) and **slicing** (taking part of a string), plus length, upper and lower case, and ASCII conversion.

Task	ERL	Python	Result for subject = "ComputerScience"

Length	subject.length	len(subject)	15
Substring (start 3, 5 characters)	subject.substring(3, 5)	subject[3:8]	"puter"
Left 4 characters	subject.left(4)	subject[:4]	"Comp"
Right 3 characters	subject.right(3)	subject[-3:]	"nce"
Uppercase	subject.upper	subject.upper()	"COMPUTERSCIENCE"
Lowercase	subject.lower	subject.lower()	"computerscience"
Character to ASCII code	ASC('A')	ord('A')	65
ASCII code to character	CHR(97)	chr(97)	'a'

In ERL, substring(x, i) means "start at index x and take i characters". In Python, [3:8] means "start at index 3 and stop **before** index 8". Both give "puter": C is index 0, o is 1, m is 2, p is 3.

Worked example: making a username with slicing and concatenation

OCR Exam Reference Language

```
firstName = input("First name")
surname = input("Surname")
part1 = surname.left(3)
part2 = firstName.left(1)
username = part1.upper + part2.upper
print("Your username is " + username)
print("It is " + str(username.length) +
" characters long")
```

Python 3

```
firstName = input("First name")
surname = input("Surname")
part1 = surname[:3]
part2 = firstName[:1]
username = part1.upper() + part2.upper()
print("Your username is " + username)
print("It is " + str(len(username)) + "
characters long")
```

With "Sam" and "Patel": part1 is "Pat", part2 is "S", so username is "PATS" and the length is 4.

Basic file handling. Programs can save data to a text file so it is kept after the program closes. The four operations are **open**, **read**, **write** and **close**. In ERL the open file must be stored in a variable, writeLine adds a line to the end of the file, and newFile creates a new empty file. Always close the file when you have finished with it.

Worked example: writing to and reading from a file

OCR Exam Reference Language	Python 3

```
newFile("scores.txt")
scoreFile = open("scores.txt")
scoreFile.writeLine("Dougie,42")
scoreFile.writeLine("Sam,37")
scoreFile.close()

scoreFile = open("scores.txt")
while NOT scoreFile.endOfFile()
    print(scoreFile.readLine())
endwhile
scoreFile.close()
```

```
scoreFile = open("scores.txt", "w")
scoreFile.write("Dougie,42\n")
scoreFile.write("Sam,37\n")
scoreFile.close()

scoreFile = open("scores.txt", "r")
for line in scoreFile:
    print(line.strip())
scoreFile.close()
```

The output is Dougie,42 then Sam,37. In Python, the second value in open() is the mode: "r" reads, "w" creates a new file (wiping any old one) and writes, "a" adds to the end. \n ends each line, and strip() removes it when printing.

Records store all the data about one thing (one student, one product) together. A record is made up of **fields**, and each field can be a different data type. A **database table** is a collection of records with the same fields: each row is a record and each column is a field.

Table: Students			
StudentID	**Name**	**Form**	**Age**
S01	Amir	10A	15
S02	Bella	10B	14
S03	Chen	10A	14
S04	Dani	10B	15
S05	Ella	10A	15

SQL (Structured Query Language) is used to search for data in a database table. You need three commands: **SELECT** (which fields to show), **FROM** (which table) and **WHERE** (which records, using a condition). SELECT * means "all fields". SQL uses a single = for "equal to", and text values go in quotes.

Worked example: SQL queries on the Students table

```
SELECT Name
FROM Students
WHERE Form = "10A"
```

Name
Amir
Chen
Ella

```
SELECT Name, Age
FROM Students
WHERE Age > 14
```

Name	Age
Amir	15
Dani	15
Ella	15

```
SELECT *
FROM Students
WHERE Form = "10B" AND Age = 14
```

StudentID	Name	Form	Age
S02	Bella	10B	14

Arrays store several values of the **same data type** under one name. Each value is found by its **index**, starting at 0. In OCR's specification arrays are **fixed length** (static): you set the size when you create the array and it does not change. Python lists can grow and shrink, but in the exam treat arrays as fixed length.

- A **1D array** is a single list: `array colours[5]` has indexes 0 to 4.
- A **2D array** is a grid with rows and columns: `array gameboard[8,8]` has 8 rows and 8 columns, each numbered 0 to 7. A 2D array can be used like a database table: each row is a record and each column is a field.

Worked example: a 1D array

OCR Exam Reference Language	Python 3

`array colours = ["Blue", "Pink", "Green", "Yellow", "Red"]`
`print(colours[2])`
`colours[0] = "Purple"`
`for i = 0 to 4`
`  print(colours[i])`
`next i`
	`colours = ["Blue", "Pink", "Green", "Yellow", "Red"]`
	`print(colours[2])`
	`colours[0] = "Purple"`
	`for i in range(0, 5):`
	`  print(colours[i])`

colours[2] is "Green" (the third item, because counting starts at 0). The loop then prints Purple, Pink, Green, Yellow, Red.

Worked example: a 2D array used as a database table

This stores the first three records of the Students table (StudentID, Name, Form). Everything is stored as a string, because an array holds one data type. Indexes are [row, column].

OCR Exam Reference Language	Python 3

```
array students[3,3]
students[0,0] = "S01"
students[0,1] = "Amir"
students[0,2] = "10A"
students[1,0] = "S02"
students[1,1] = "Bella"
students[1,2] = "10B"
students[2,0] = "S03"
students[2,1] = "Chen"
students[2,2] = "10A"

for row = 0 to 2
    if students[row,2] == "10A" then
        print(students[row,1])
    endif
next row
```

```
students = [["S01", "Amir", "10A"],
            ["S02", "Bella", "10B"],
            ["S03", "Chen", "10A"]]

for row in range(0, 3):
    if students[row][2] == "10A":
        print(students[row][1])
```

The loop checks the Form field (column 2) of each record and prints the Name (column 1): Amir, then Chen. It does the same job as SELECT Name FROM Students WHERE Form = "10A". Note that ERL writes students[row,2] and Python writes students[row][2].

Sub programs are named blocks of code that you write once and call (run) whenever you need them. They produce structured code: the program is split into parts that are easier to read, test, reuse and share between programmers.

- A **procedure** carries out a task but does not return a value.
- A **function** carries out a task and **returns** a value. Store the returned value in a variable if you need it later, or use it straight away, for example in print().
- **Parameters** are the values passed into a sub program when it is called. Arrays can be passed in as parameters and returned from functions.

Use a function when you need an answer back (such as a total or a result of a check). Use a procedure when you just need something done (such as printing a menu).

Worked example: a procedure and a function

OCR Exam Reference Language

```
procedure greet(name)
    print("Hello " + name)
endprocedure

function squared(number)
    squared = number^2
    return squared
endfunction

greet("Dougie")
newValue = squared(4)
print(newValue)
```

Python 3

```
def greet(name):
    print("Hello " + name)

def squared(number):
    result = number ** 2
    return result

greet("Dougie")
newValue = squared(4)
print(newValue)
```

The output is "Hello Dougie" then 16. In Python a sub program is written with def, whether it is a function or a procedure.

Worked example: passing an array into a function and returning an array

OCR Exam Reference Language

```
function total(scores)
    sum = 0
    for i = 0 to 4
        sum = sum + scores[i]
    next i
    return sum
endfunction

function doubled(scores)
    array newScores[5]
    for i = 0 to 4
        newScores[i] = scores[i] * 2
    next i
    return newScores
endfunction

array myScores = [4, 7, 2, 9, 3]
print(total(myScores))
bigScores = doubled(myScores)
print(bigScores[0])
```

Python 3

```
def total(scores):
    sum = 0
    for i in range(0, 5):
        sum = sum + scores[i]
    return sum

def doubled(scores):
    newScores = [0, 0, 0, 0, 0]
    for i in range(0, 5):
        newScores[i] = scores[i] * 2
    return newScores

myScores = [4, 7, 2, 9, 3]
print(total(myScores))
bigScores = doubled(myScores)
print(bigScores[0])
```

total returns 4 + 7 + 2 + 9 + 3 = 25. doubled returns a new array [8, 14, 4, 18, 6], so bigScores[0] is 8.

Local and global variables. A **local** variable (or constant) is created inside a sub program and can only be used there; it disappears when the sub program ends. A **global** variable (or constant) is created in the main program and can be used anywhere. Local variables are safer: they cannot be changed by mistake from another part of the program, and two sub programs can use the same name without clashing. Global variables are useful for values the whole program needs, such as a score.

Worked example: local and global variables

OCR Exam Reference Language

```
global score = 0

procedure addPoints(points)
    const bonus = 5
    score = score + points + bonus
endprocedure

addPoints(10)
print(score)
```

Python 3

```
score = 0

def addPoints(points):
    global score
    BONUS = 5
    score = score + points + BONUS

addPoints(10)
print(score)
```

score is global, so the procedure can change it. bonus is a local constant: it only exists inside addPoints. The output is 15. In Python, a sub program must say global score before it can change a global variable.

Random numbers. Programs can create random numbers, for example for dice, games and quizzes. In ERL, random(1,6) gives a random integer from 1 to 6, including both 1 and 6. random(-1.0,10.0) gives a random real number from -1.0 to 10.0.

Worked example: a guessing game with a random number

OCR Exam Reference Language

```
secret = random(1, 10)
do
    guess = int(input("Guess a number
from 1 to 10"))
    if guess < secret then
        print("Too low")
    elseif guess > secret then
        print("Too high")
    endif
until guess == secret
print("Correct")
```

Python 3

```
import random
secret = random.randint(1, 10)
while True:
    guess = int(input("Guess a number
from 1 to 10"))
    if guess < secret:
        print("Too low")
    elif guess > secret:
        print("Too high")
    if guess == secret:
        break
print("Correct")
```

In Python, import random first. random.randint(1, 10) includes both 1 and 10. For a random real number, use random.uniform(-1.0, 10.0).

Key terms

Data type
The kind of data a variable holds, such as integer or string.

Integer
A whole number, positive, negative or zero.

Real
A number with a decimal part (called float in Python).

Boolean
A data type with only two values, True or False.

Character
A single letter, digit or symbol.

String
A sequence of characters, such as a name.

Casting
Temporarily changing a value from one data type to another, such as with int() or str().

Concatenation
Joining strings together using +.

Slicing
Taking part of a string, such as the first three characters.

ASCII
A character set that gives each character a number, such as 65 for 'A'.

File handling
Opening, reading from, writing to and closing a file from inside a program.

Record

All the data about one item, made up of fields that can be different data types.

Field
One piece of data in a record, such as Name or Age; a column in a table.

Database table
A collection of records with the same fields, shown as rows and columns.

SQL
A language used to search for data in a database table.

SELECT
The SQL command that says which fields to show.

FROM
The SQL command that says which table to search.

WHERE
The SQL command that gives the condition records must meet.

Array
A fixed-length structure that stores several values of the same data type under one name.

Index
The position number of an item in an array or string, starting at 0.

1D array
An array that is a single list of values.

2D array
An array arranged as a grid of rows and columns, using two indexes.

Sub program
A named block of code that can be called from elsewhere in the program.

Procedure
A sub program that carries out a task but does not return a value.

Function
A sub program that carries out a task and returns a value.

Parameter
A value passed into a sub program when it is called.

Return value
The value a function sends back to the code that called it.

Local variable
A variable created inside a sub program that can only be used inside it.

Global variable
A variable created in the main program that can be used anywhere in the program.

Random number generation
Making a program pick a number by chance within a given range.

Exam mistakes that lose marks

- Joining a number to a string without casting: write `"Score: " + str(score)`.
- Reading substring(x, i) as start and end. The second number is how many characters to take.
- Forgetting that strings and arrays start at index 0, so the last index of a 5-item array is 4.

- Storing a phone number as an integer, which loses the leading 0.
- Opening a file without storing it in a variable, or forgetting to close it.
- SQL slips: using == in WHERE, leaving out FROM, forgetting quotes around text values, or listing fields the question did not ask for.
- Getting row and column the wrong way round in a 2D array. Use the order the question gives you.
- Writing a function that never returns a value, or calling a function and not using or storing what it returns.
- Saying a procedure returns a value. Only functions return values.
- Thinking random(1, 6) cannot give 6. Both ends are included.

Quick check

1. Which data type should store a phone number, and why?
2. What does word.substring(1, 3) return if word = "Science"?
3. Write an SQL query to show the Name of every student in form 10B from the Students table.
4. What is the difference between a function and a procedure?
5. What is a local variable?

▼ **Answers**

1. String, because it may start with 0, may contain spaces, and is never used in calculations.
2. "cie" (start at index 1, take 3 characters: c, i, e).
3. SELECT Name FROM Students WHERE Form = "10B" (gives Bella and Dani).
4. A function returns a value; a procedure does not.
5. A variable created inside a sub program that can only be used inside that sub program.

2.3 Producing robust programs

Specification points 2.3.1 Defensive design and 2.3.2 Testing.

What you need to know

Defensive design means planning a program so that it keeps working whatever the user does. The programmer must think about every likely input, and decide how the program deals with invalid data.

Anticipating misuse means predicting how users might break the program, by mistake or on purpose, and stopping it. Examples: typing letters where a number is expected; leaving an input blank; entering a number that is far too big; dividing by zero; guessing passwords again and again (stop this by limiting the number of attempts).

Authentication confirms the identity of a user, for example with a username and password. Only people who prove who they are get access.

Worked example: simple authentication with a limit of 3 attempts

OCR Exam Reference Language

```
attempts = 0
do
    username = input("Enter username")
    password = input("Enter password")
    attempts = attempts + 1
until (username == "dougie" AND password
== "Blue42") OR attempts == 3

if username == "dougie" AND password ==
"Blue42" then
    print("Access granted")
else
    print("Account locked")
endif
```

Python 3

```
attempts = 0
while True:
    username = input("Enter username")
    password = input("Enter password")
    attempts = attempts + 1
    if (username == "dougie" and
password == "Blue42") or attempts == 3:
        break

if username == "dougie" and password ==
"Blue42":
    print("Access granted")
else:
    print("Account locked")
```

The loop stops when the details are correct or after the third try, whichever comes first. Limiting attempts stops someone guessing passwords over and over.

Input validation checks that data entered is sensible and allowed before the program uses it. If it is not, the program rejects it and asks again. Validation cannot check that data is correct (someone can type a valid but wrong age); it checks the data follows the rules. Common checks:

Check	What it does	Example
Range check	The value is within set limits	Age is from 11 to 16
Type check	The value is the right data type	Age is an integer, not "fifteen"
Length check	The value has an allowed number of characters	Password is at least 8 characters
Presence check	Something has been entered	Name is not left blank

Format check	The value follows a pattern	A date is written as DD/MM/YYYY

Worked example: a range check with a loop

OCR Exam Reference Language

```
age = int(input("Enter your age (11 to 16)"))
while age < 11 OR age > 16
    print("Age must be from 11 to 16")
    age = int(input("Enter your age (11 to 16)"))
endwhile
print("Age accepted")
```

Python 3

```
age = int(input("Enter your age (11 to 16)"))
while age < 11 or age > 16:
    print("Age must be from 11 to 16")
    age = int(input("Enter your age (11 to 16)"))
print("Age accepted")
```

The loop keeps asking until the age is in range. The condition uses OR because an age is wrong if it is too low or too high. This code still has a weakness: if the user types "fifteen", int() cannot convert it and the program stops with an error. A fully robust program also needs a type check before casting.

Maintainability means writing code that is easy for you or another programmer to read, fix and update later. Four ways:

- **Sub programs**: split code into named procedures and functions, so each part can be understood, tested and reused on its own.
- **Naming conventions**: give variables and sub programs meaningful names (totalScore, not ts) and use one style throughout.
- **Indentation**: indent the code inside each selection, loop and sub program, so you can see where each block starts and ends.
- **Commenting**: add notes that explain what a section of code does and why. Comments are ignored when the program runs. They help others (and you, months later) understand the code. Good comments explain purpose; they do not just repeat the line.

Worked example: hard to maintain

OCR Exam Reference Language

```
a = float(input())
b = a * 0.2
print(a + b)
```

Python 3

```
a = float(input())
b = a * 0.2
print(a + b)
```

The same program, easy to maintain

OCR Exam Reference Language

Python 3

```
//Works out the price of an item
including VAT

function addVat(price)
    //Returns the price with 20% VAT
added
    const vat = 0.2
    return price + price * vat
endfunction

price = float(input("Enter the price"))
total = addVat(price)
print("Total with VAT: " + str(total))
```

```
#Works out the price of an item
including VAT

def addVat(price):
    #Returns the price with 20% VAT
added
    VAT = 0.2
    return price + price * VAT

price = float(input("Enter the price"))
total = addVat(price)
print("Total with VAT: " + str(total))
```

The second version has comments, meaningful names, a constant instead of an unexplained 0.2, a function, clear indentation and a prompt that tells the user what to type.

The purpose of testing is to find errors and make sure the program does what it is meant to do, works for all likely inputs and does not crash. Testing can show errors are present; it cannot prove there are none.

- **Iterative testing** happens during development. Each module (part) of the program is tested as it is written, fixed and tested again.
- **Final (terminal) testing** happens at the end of production. The whole program is tested together to check every part works with the others and that it meets the requirements.

Syntax and logic errors

- A **syntax error** breaks the grammatical rules of the programming language. It stops the program being run or translated. Examples: a missing bracket or quote, a misspelt keyword, a missing then or endif.
- A **logic error** lets the program run but gives unexpected output. Examples: the wrong operator, a calculation in the wrong order, a loop that runs the wrong number of times.

Worked example: a syntax error

OCR Exam Reference Language

```
score = int(input("Enter score"))
if score >= 50
    print("Pass)
endif
```

Python 3

```
score = int(input("Enter score"))
if score >= 50
    print("Pass)
```

Two syntax errors: the if line is missing then (in Python, the colon), and "Pass is missing its closing quote. Fix: `if score >= 50 then` (Python: `if score >= 50:`) and `print("Pass").`

A logic error

OCR Exam Reference Language

```
num1 = 6
num2 = 4
average = num1 + num2 / 2
print(average)
```

Python 3

```
num1 = 6
num2 = 4
average = num1 + num2 / 2
print(average)
```

This runs, but outputs 8 instead of 5, because division happens before addition (6 + 2 = 8). Fix: `average = (num1 + num2) / 2`.

Test data. You need to choose suitable test data for a scenario. OCR defines four types:

- **Normal**: data which should be accepted by the program without causing errors.
- **Boundary**: data of the correct type which is on the very edge of being valid.
- **Invalid**: data of the correct data type which should be rejected.
- **Erroneous**: data of the incorrect data type which should be rejected.

The difference between invalid and erroneous is the **data type**. For an age that must be an integer from 11 to 16, 25 is invalid (it is an integer, but out of range) and "fifteen" is erroneous (it is not an integer at all).

Test plans. A test plan lists each test before it is carried out: what is being tested, the test data, the type of test and the expected result. The actual result is filled in when the test is run. You need to be able to create and complete one.

Worked example: test plan for "Enter your age (11 to 16)", which must be a whole number

Test	Test data	Type	Expected result
1	13	Normal	Accepted
2	11	Boundary	Accepted
3	16	Boundary	Accepted
4	10	Invalid	Rejected with a message; asked to enter again
5	25	Invalid	Rejected with a message; asked to enter again
6	"fifteen"	Erroneous	Rejected with a message; asked to enter again
7	13.5	Erroneous	Rejected with a message; asked to enter again

11 and 16 are the boundary values because they are the very edge of being valid. 10 and 17 are just outside the boundary, so they are invalid and should be rejected. 13.5 is erroneous because it is a real number, not an integer.

Refining algorithms means improving them: fixing errors found in testing, making them more efficient, or adding features such as validation. The range check example above is a refinement: the first version of the program just accepted any age. To refine it, the programmer added a while loop around the input. In the exam, a "refine" question gives you code and asks you to change it; write the changed code in ERL or Python, not a description.

Key terms

Defensive design
Planning a program so that it copes with any likely input and keeps working.

Anticipating misuse
Predicting how users might break a program and preventing it.

Authentication

Confirming the identity of a user, for example with a username and password.

Input validation
Checking that input is sensible and allowed before the program uses it.

Range check
A validation check that a value lies between set limits.

Type check
A validation check that a value is the correct data type.

Length check
A validation check that a value has an allowed number of characters.

Presence check
A validation check that something has been entered.

Format check
A validation check that a value follows a set pattern.

Maintainability
How easy a program is to read, fix and update.

Naming conventions
Rules for giving variables and sub programs clear, consistent names.

Indentation
Moving code inwards to show which lines belong inside a block.

Comment
A note in the code that explains it and is ignored when the program runs.

Iterative testing
Testing each module during development, fixing it and testing again.

Final (terminal) testing
Testing the whole program at the end of production.

Syntax error
An error that breaks the grammatical rules of the language and stops the program being run or translated.

Logic error
An error that lets the program run but produces unexpected output.

Normal test data
Data that should be accepted by the program without causing errors.

Boundary test data
Data of the correct type on the very edge of being valid.

Invalid test data
Data of the correct data type that should be rejected.

Erroneous test data
Data of the incorrect data type that should be rejected.

Test plan
A table of planned tests showing the test data, type of test, expected result and actual result.

Refining
Improving an algorithm by fixing errors, making it more efficient or adding features.

Exam mistakes that lose marks

- Mixing up invalid and erroneous data. Invalid is the right type but should be rejected (25 for an age of 11 to 16); erroneous is the wrong type ("fifteen").
- Giving boundary data from the middle of the range. Boundary data is on the very edge: for 11 to 16, use 11 and 16.
- Writing "it works" as an expected result. Say exactly what should happen: "accepted" or "rejected with an error message".
- Confusing validation with authentication. Validation checks data follows the rules; authentication checks who the user is.
- Saying validation makes sure data is correct. It only checks that data is sensible and allowed.
- Calling a logic error a syntax error. If the program runs but gives the wrong answer, it is a logic error.
- Writing comments that just repeat the code instead of explaining what it is for.
- Saying testing proves a program has no errors.
- Describing a fix in words when the question says refine or rewrite the code.

Quick check

1. A program accepts whole numbers from 1 to 100. Give the two boundary values.
2. For the same program, give one invalid and one erroneous test value.
3. A programmer forgets to write endif. Is this a syntax error or a logic error?
4. Give two ways to make a program easier to maintain.
5. What is the difference between iterative and final testing?

▼ **Answers**

1. 1 and 100.
2. Invalid: 150 (or 0, or -5). Erroneous: "ten" (or 12.5).
3. A syntax error: it breaks the rules of the language, so the program will not run.
4. Any two of: comments, meaningful variable names, indentation, sub programs.
5. Iterative testing tests each module during development; final testing tests the whole program at the end of production.

2.4 Boolean logic

Specification point 2.4.1 Boolean logic.

What you need to know

Computers work with two states: 1 (true, on) and 0 (false, off). **Logic gates** take one or more inputs of 1 or 0 and give one output. A **truth table** lists every possible combination of inputs and the output for each. You need to know the truth table and the symbol for each of the three gates, and be able to create, complete and edit logic diagrams and truth tables with more than one gate.

OCR accepts other valid notation in the exam, for example T and F instead of 1 and 0, or V for OR.

Gate symbols (you must recognise and draw these):

Gate	Also called	Shape of the symbol	Inputs
AND	Conjunction	A "D" shape: flat straight back where the inputs go in, round front where the output comes out	2
OR	Disjunction	A curved back where the inputs go in, sweeping to a point at the front where the output comes out (like an arrowhead or shield)	2
NOT	Negation	A triangle pointing towards the output, with a small circle at its tip	1

The three truth tables

AND: the output is 1 only when both inputs are 1

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

OR: the output is 1 when at least one input is 1

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

NOT: the output is the opposite of the input

A	NOT A
0	1

1	0

Combining gates. A logic diagram can join gates together: the output of one gate becomes an input to the next. With 2 inputs there are 4 rows in the truth table; with 3 inputs there are 8 rows. List the inputs by counting up in binary (000, 001, 010 up to 111) so you never miss a row. Add a column for each gate's output, working from the inputs towards the final output, and do anything in brackets first.

Worked example: Q = (A AND B) OR NOT C

The logic diagram, in words: A and B go into an AND gate. C goes into a NOT gate. The output of the AND gate and the output of the NOT gate both go into an OR gate. The output of the OR gate is Q.

A	B	C	A AND B	NOT C	Q = (A AND B) OR NOT C
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

How to read it: Q is 1 whenever C is 0 (because NOT C is then 1), and also when A and B are both 1. The only rows where Q is 0 are where C is 1 and A AND B is 0.

Applying logic to problems. Turn the words of a scenario into a Boolean expression: "and" becomes AND, "or" becomes OR, "not" becomes NOT. Then use a truth table to check every case.

Worked example: a ride entry check

"You can go on the ride if you are at least 12 AND at least 140 cm tall, OR if you are with an adult." Let A = at least 12, B = at least 140 cm, C = with an adult. Then: allowed = (A AND B) OR C.

OCR Exam Reference Language	Python 3

```
age = int(input("Enter your age"))
height = int(input("Enter your height in
cm"))
withAdult = input("With an adult?
(yes/no)")
if (age >= 12 AND height >= 140) OR
withAdult == "yes" then
    print("You can ride")
else
    print("Sorry, you cannot ride")
endif
```

```
age = int(input("Enter your age"))
height = int(input("Enter your height in
cm"))
withAdult = input("With an adult?
(yes/no)")
if (age >= 12 and height >= 140) or
withAdult == "yes":
    print("You can ride")
else:
    print("Sorry, you cannot ride")
```

A 10-year-old who is 150 cm tall: A = 0, B = 1, so A AND B = 0. If they are with an adult, C = 1, so the result is 0 OR 1 = 1 and they can ride. Without an adult, the result is 0 OR 0 = 0.

Key terms

Boolean logic
Logic that works with only two values, 1 (true) and 0 (false).

Logic gate
A component that takes one or more inputs of 1 or 0 and gives a single output.

AND gate (conjunction)
A gate whose output is 1 only when both inputs are 1.

OR gate (disjunction)
A gate whose output is 1 when at least one input is 1.

NOT gate (negation)
A gate with one input whose output is the opposite of the input.

Truth table
A table showing every possible combination of inputs and the output for each.

Logic diagram
A drawing of logic gates joined together to show how inputs produce an output.

Boolean expression
A statement such as (A AND B) OR NOT C that combines inputs using AND, OR and NOT.

Exam mistakes that lose marks

- Missing rows in a truth table. 2 inputs need 4 rows; 3 inputs need 8. Count up in binary.
- Thinking OR gives 0 when both inputs are 1. OR gives 1 when one or both inputs are 1.
- Drawing a NOT gate without the small circle, or mixing up the AND and OR shapes. AND has a flat back; OR has a curved back and a pointed front.
- Ignoring the brackets. Work out the part in brackets first, then the rest.
- Applying NOT to the wrong part: NOT C only flips C, not the whole expression.
- Skipping the middle columns. Writing the output of each gate makes mistakes less likely and can earn marks.
- Changing the order of inputs given in the question's table. Fill in the table exactly as it is laid out.

Quick check

1. What is NOT 1?
2. What are 1 AND 0, and 1 OR 0?
3. How many rows does a truth table with 3 inputs need?
4. For Q = (A OR B) AND NOT C, what is Q when A = 1, B = 0 and C = 1?
5. A light L comes on if switch S is on AND either it is dark D OR the override O is on. Write the Boolean expression.

▼ **Answers**

1. 0.
2. 1 AND 0 = 0. 1 OR 0 = 1.
3. 8.
4. A OR B = 1. NOT C = 0. 1 AND 0 = 0, so Q = 0.
5. L = S AND (D OR O).

2.5 Programming languages and IDEs

Specification points 2.5.1 Languages and 2.5.2 The Integrated Development Environment (IDE).

What you need to know

High-level and low-level languages

	High-level languages (such as Python)	Low-level languages (machine code and assembly language)
What they look like	English-like words and maths symbols, such as print and if	Machine code is binary (1s and 0s). Assembly language uses short codes for each processor instruction.
Ease of use	Easier to read, write, understand and debug	Hard to read, write and debug
Instructions	One line can stand for many machine code instructions	Each instruction is usually one processor instruction
Portability	Portable: the same code can be translated to run on different types of processor	Written for one type of processor, so it does not work on others
Translation	Must be translated into machine code before the processor can run it	Machine code runs directly; assembly language must be translated
Control and speed	Less direct control of the hardware	Direct control of the hardware and memory; code can be very fast and small
Used for	Most programs and apps	Programs that must be fast or small, or control hardware directly, such as device drivers or embedded systems

Why translators are needed. A processor can only run machine code. Programs written in a high-level language (the **source code**) must be translated into machine code first. A **translator** does this. The two you need to know are compilers and interpreters.

	Compiler	Interpreter
How it translates	Translates the whole program in one go, before it runs	Translates and runs the program one line at a time
What it produces	An executable file of machine code that can be run again and again	No executable file; it translates the code every time it is run
Errors	Reports the errors it finds after trying to translate the whole program; no executable is made until they are fixed	Stops at the first error and shows which line it is on
Speed when running	The compiled program runs quickly, because it is already translated	Runs more slowly, because each line is translated as it goes

Sharing the program	Users run the executable without needing the compiler, and they do not see the source code	Users need the interpreter and the source code to run it
Best for	Finished programs that are given to users	Developing and testing, because you can run code straight away and find errors line by line

Worked example: the same program, two translators

A 50-line program has a syntax error on line 20.

- **Interpreter**: lines 1 to 19 run, then the interpreter stops at line 20 and reports the error on that line. You fix it and run again straight away.
- **Compiler**: nothing runs. The compiler translates the whole program, reports the error on line 20 (and any other errors it finds), and does not make an executable file until every error is fixed. Once it compiles, the executable runs fast and can be given to users without the source code.

The Integrated Development Environment (IDE) is software that gives a programmer the tools they need to write, run, test and debug programs in one place. You need to know the tools and explain how each one helps a programmer develop a program.

Tool	What it does	How it helps the programmer
Editor	Where you type and change code. Features include syntax highlighting (colouring keywords), automatic indentation, auto-complete and line numbers.	Code is quicker to write and easier to read; colouring and line numbers help you spot mistakes and find lines.
Error diagnostics	Finds errors and reports them, often with the line number and a message saying what is wrong. Debugging tools can let you stop the program at a chosen line (a breakpoint), run it one line at a time and watch the values of variables.	Errors are found and fixed faster, and you can see exactly where a logic error happens.
Run-time environment	Lets you run the program inside the IDE and see the output, without leaving it.	You can test the program straight away while you are writing it; if it crashes, the IDE can show where.
Translator	A built-in compiler or interpreter that turns the source code into machine code.	You do not need separate software to translate and run your code.

Key terms

High-level language

A programming language that uses English-like words, is portable and must be translated before it runs.

Low-level language
A language close to what the processor understands, such as machine code or assembly language.

Machine code
Instructions in binary that the processor can run directly.

Assembly language
A low-level language that uses short codes to stand for processor instructions.

Source code
The program as the programmer wrote it, before translation.

Translator
A program that converts source code into machine code.

Compiler
A translator that converts the whole program into an executable machine code file in one go.

Interpreter
A translator that translates and runs a program one line at a time.

Executable file
A file of machine code that can be run without the source code or the translator.

Portable
Able to run on different types of processor once translated.

Integrated Development Environment (IDE)
Software that gives programmers the tools to write, run, test and debug code in one place.

Editor
The part of an IDE where code is typed and changed.

Error diagnostics
IDE tools that find and report errors to help the programmer fix them.

Run-time environment
The part of an IDE that lets a program be run and tested inside it.

Breakpoint
A point set in the code where the program pauses so the programmer can check variable values.

Exam mistakes that lose marks

- Saying a compiler translates one line at a time. That is an interpreter.
- Saying an interpreter makes an executable file. Only a compiler does.
- Saying high-level programs run faster than low-level ones, or that low-level languages are easier to write.
- Saying low-level code works on any processor. It is written for one type of processor.
- Listing IDE tools without explaining how each one helps the programmer, when the question asks how or why.
- Answering "it makes it easier" with no detail. Say what is easier and why, for example "line numbers in error messages show exactly where to look".
- Writing that a compiler stops at the first error. A compiler reports the errors it finds after going through the whole program; an interpreter stops at the first one.

Quick check

1. Why are translators needed?
2. Which translator translates the whole program in one go?
3. Give one reason an interpreter is useful while a program is being developed.
4. Name two features of an IDE editor.
5. Give one reason a programmer might choose a low-level language.

▼ **Answers**

1. Because the processor can only run machine code, so source code in a high-level language must be translated first.
2. A compiler.
3. It stops at the line with the error and shows where it is, and the code can be run straight away without compiling.
4. Any two of: syntax highlighting, automatic indentation, auto-complete, line numbers.
5. It gives direct control of the hardware, and the code can be very fast and use little memory.